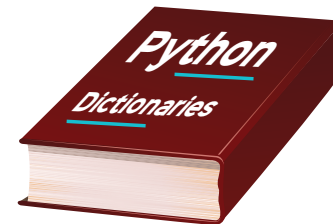


Engaging with Dictionaries and Sets in Python



In this first article in a two-part series, we will delve into the fascinating world of dictionaries in Python and uncover some insights that are sure to captivate your interest.

In Python, dictionaries and sets are used to save unique, unordered, mutable (also immutable — frozen sets) and quick retrieval elements. In any Python program there are a number of dicts used even when the program is not using these explicitly; e.g., CPython uses dictionary lookup for any attribute, class or global variable and more. As this plays an important role, it has been highly optimised by Python's core developers, especially for search, add and delete operations. Hash tables are the key element behind the implementation of dicts and sets, making these data structures efficient. These tables are powerful and are also used in solving problems like indexing of database tables, caching, name lookups and so on.

A dictionary is composed of a series of key-value mapping elements. The keys and values can be of mixed types like string and integer.

Create

Here are a few methods to create a dict:

- `a = dict(one=1, two=2, three=3)`
- `b = {'one': 1, 'two': 2, 'three': 3}`
- `c = dict(zip(['one', 'two', 'three'], [1, 2, 3]))`
- `d = dict([('one', 1), ('two', 2), ('three', 3)])`
- `e = dict({'one': 1, 'two': 2, 'three': 3})`

Possible key types are all the immutable types such as str, bytes, and numeric.

Update

The `update()` method inserts the specified items to the dictionary. These items can be a dictionary or an iterable object with key value pairs. The `pop()` method removes the specified item from the dictionary. The value of the removed item is the return value of the `pop()` method.

```
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
print("Pre Removed dict:", car)
x = car.pop("model")
```

```
print("Removed value:", x)
print("Post Removed dict:", car)
```

Output:

```
Pre Removed dict: {'brand': 'Ford', 'model': 'Mustang',
'year': 1964}
Removed value: Mustang
Post Removed dict: {'brand': 'Ford', 'year': 1964}
```

Get

This returns the default value or *None* if the key does not exist, rather than handling `KeyError`.

The `__missing__` method is called by the `__getitem__()` dictionary method internally if the keys don't exist. The return value of `__missing__()` is the value returned when accessing the non-existence key.

Here's an example:

```
sample_dict = {'name': 'python', 'age': 20}
print(sample_dict['location'])
Traceback (most recent call last):
  File "dict_set_sample.py", line 40, in <module>
    print(sample_dict['location'])
KeyError: 'location'
sample_dict = {'name': 'python', 'age': 20}
print(sample_dict.get('location'))
print(sample_dict.get('location', 'India'))
```

Output:

```
None
India
```

Items

The `items()` method returns a view object. The view object contains the key-value pairs of the dictionary as tuples in a list, and will reflect any changes done to the dictionary.

 **Note:** 'Returned view object' was part of version 3+, and the `__contains__` methods defined in Python dict class has been introduced to find out if the key exists in the dictionary or sets.